WIFI-STA Mode

1. WIFI-STA Mode Overview

In STA (Station) mode, ESP32-S3 acts as a Wi-Fi client to connect to an existing router or hotspot. After obtaining an IP address, it can access the internet. This experiment demonstrates a Wi-Fi connection algorithm with timeout detection and automatic retry, which serves as the foundation for all subsequent network experiments (UDP/TCP/HTTP/MQTT).

Typical Application Scenarios:

Application Type	Example
IoT Devices	Smart outlets, sensor nodes connecting to the internet via router
Data Reporting	Temperature and humidity sensors periodically pushing data to cloud platform
Remote Control	Mobile app controlling ESP32 peripherals through cloud
OTA Updates	Downloading new firmware from HTTP server

2. Core Concepts

2.1 Wi-Fi Connection Process

`WiFi.begin(ssid, pass)` initiates an 802.11 connection, with the underlying steps: scanning target AP → Authentication → Association → WPA2 four-way handshake to establish encrypted link → DHCP to obtain IP address.

2.2 Connection Status Codes

`WiFi.status()` return values:

Status Code	Meaning
`WL_CONNECTED` (3)	Connected and IP obtained
`WL_DISCONNECTED` (6)	Not associated with any AP
`WL_CONNECTION_LOST` (5)	Link lost after association
`WL_CONNECT_FAILED` (4)	Wrong password or authentication failed
`WL_NO_SSID_AVAIL` (1)	Target SSID not found during scan

2.3 Reconnection Algorithm with Timeout

In environments with poor signal, `WiFi.begin()` may block for a long time. This experiment uses `millis()` to implement a 10-second timeout timer. After timeout, it waits 15 seconds before retrying to avoid overwhelming the router with frequent requests.

---

3. Hardware Dependencies

Pure software experiment. ESP32-S3 has a built-in 2.4GHz Wi-Fi transceiver, supporting 802.11 b/g/n (Wi-Fi 4).

---

4. Project Architecture and Flow

```
setup()
  ├─ Serial.begin(115200)
  ├─ WiFi.mode(WIFI_STA)       // Set to pure STA mode
  └─ connectWiFi()             // Connect to router (with timeout retry)

connectWiFi() Algorithm:
  while(true):
    WiFi.disconnect(true)      // Disconnect and clear credentials stored in NVS
    WiFi.begin(SSID, PASS)     // Initiate 802.11 connection
    while(status != CONNECTED && not timeout):
      delay(500)
    If connected → Print IP → return
    If timeout → delay(15000) → retry
```

5. Source Code Details

Source code: `Source code → Arduino → 3.Network Communication → WIFI_STA → WIFI_STA.ino`

5.1 Configuration Section

```
#include <WiFi.h>
const char* WIFI_SSID = "Yahboom2";
const char* WIFI_PASS = "yahboom890729";
```

5.2 `connectWiFi()` — Connection Algorithm with Timeout

```
void connectWiFi() {
  const int TIMEOUT_MS = 10000;  // 10 second timeout

  int attempt = 0;
  while (true) {
    attempt++;
    Serial.printf("Connecting to WiFi: %s (Attempt %d)\n", WIFI_SSID, attempt);

    WiFi.disconnect(true);   // true = also clear WiFi credentials saved in NVS
    delay(500);
    WiFi.begin(WIFI_SSID, WIFI_PASS);

    unsigned long startTime = millis();
    while (WiFi.status() != WL_CONNECTED && (millis() - startTime) < TIMEOUT_MS) {
      delay(500);
      Serial.print(".");
    }

    if (WiFi.status() == WL_CONNECTED) {
      Serial.println("\nWiFi Connection succeeded!");
      Serial.print("Device IP: ");
      Serial.println(WiFi.localIP());
      return;
    }

    Serial.println("\nWiFi Connection timed out! Retrying in 15s...");
    delay(15000);
  }
}
```

> The `true` parameter in `WiFi.disconnect(true)` also erases credentials stored in NVS, ensuring each retry starts from a clean state. `WiFi.localIP()` returns the IPv4 address assigned by the DHCP server.

5.3 `setup()` — Startup Entry Point

```
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);    // Pure STA mode — does not create AP signal, saves power
  connectWiFi();
}

void loop() { delay(1000); }
```

> `WiFi.mode(WIFI_STA)` sets pure client mode. The counterparts are `WIFI_AP` (access point) and `WIFI_AP_STA` (dual-mode coexistence). Pure STA saves power and does not generate interfering AP signals.

---

6. Operation Flow

6.1 Build and Flash

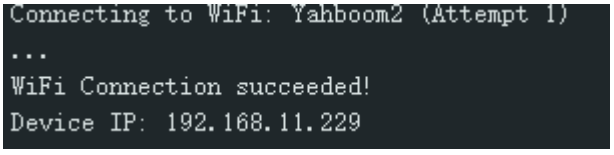
Arduino flashing process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Modify `WIFI_SSID` and `WIFI_PASS` to your router credentials
2. Build and flash

6.2 Expected Behavior

Open the serial monitor (115200 bps). After successful connection, it prints:

```
Connecting to WiFi: Yahboom2 (Attempt 1)
...
WiFi Connection succeeded!
Device IP: 192.168.11.229
```



6.3 Verification Methods

Action	Expected Result
Enter correct password	Prints IP address, WiFi connection succeeds
Enter wrong password	Retries every 15 seconds after timeout, status code 4 (WL_CONNECT_FAILED)
Change timeout to 5 seconds	Triggers retry faster under weak signal

---

7. Common Issues

Symptom	Cause	Solution
Continuously prints `.....` without stopping	Wrong password or signal too weak	Check password, move closer to router
Compilation error: WiFi.h not found	ESP32 board package not installed	Install ESP32 Arduino Core in IDE
`WiFi.status()` returns 1	SSID name incorrect, cannot be found during scan	Check if SSID matches router broadcast name exactly